

cs-mail Desktop MVP and CLI Administration

AI-Agent Milestone and Build Plan

C-SQD

Draft 0.1 — September 2026

Abstract

This document turns the existing *cs-mail* protocol implementation into an ordered build guide for a working desktop minimum viable product, an operator administration CLI, and a reusable Rust client core suitable for later native mobile applications. It fixes the MVP product decisions, dependency direction, stability constraints, milestone scope, acceptance tests, live demonstrations, and stop conditions. The intended reader is an AI coding agent working under human review. The plan therefore distinguishes decisions that are already made from implementation details the agent may choose, and it defines evidence of completion rather than treating code production as completion.

Status. This is a nonnormative product and implementation plan. The *cs-mail Protocol Specification* controls protocol meaning. The *Rust Reference Architecture* controls the intended implementation shape where it is more specific than this plan. No milestone in this document authorizes a silent change to settlement, wire, privacy, or security semantics.

Contents

1	How to Use This Plan	3
1.1	Source-of-truth order	3
1.2	Meaning of completion	3
2	Decisions Already Made	4
3	Architecture and Stability Rules	4
3.1	The hybrid organization	4
3.2	Dependency direction	5
3.3	Vertical modules	5
3.4	Constraint on existing crates	5
3.5	When to create another crate	6
4	Cross-Cutting Engineering Contracts	6
4.1	Security and trust	6
4.2	API and versioning	6
4.3	Client-core platform ports	7
4.4	Local data	8
4.5	Observability	8
4.6	Repository-wide quality gate	8

5 Milestone Map	8
6 Milestone 0: Protect the Foundation	9
7 Milestone 1: The Service Comes Alive	9
8 Milestone 2: Alice Gets an Identity	11
9 Milestone 3: Alice Makes Encrypted First Contact	12
10 Milestone 4: Bob's Decision Matters	13
11 Milestone 5: Time and Failure Become Real	14
12 Milestone 6: A Small Pilot Can Use It	15
13 Milestone 7: The Core Is Ready for Mobile	17
14 Agent Execution Protocol	18
14.1 Required loop for each work package	18
14.2 Prohibited shortcuts	18
14.3 Decision record threshold	18
15 End-to-End Acceptance Matrix	19
16 Risk Register and Stop Conditions	19
17 MVP Completion Definition	20
18 Recommended First Agent Assignment	20

1 How to Use This Plan

The plan follows one product story. An operator starts a service and creates a small invite-only deployment. Alice enrolls through a desktop application and receives test credits. Bob enrolls independently. Alice makes encrypted bonded first contact. Bob accepts, rejects, or blocks the solicitation. Their next interaction reflects that decision. The system then survives restarts, retries, expiry, and an offline recipient. Finally, the platform-neutral Rust client core is exposed through a narrow mobile binding without rewriting protocol logic.

Each milestone is a complete vertical increment. An agent must implement it in the listed order unless a human explicitly changes the dependency graph. Every milestone contains seven elements:

1. the user-visible promise;
2. the allowed architectural change;
3. ordered work packages;
4. mandatory tests and failure cases;
5. a live demonstration script;
6. exit evidence; and
7. work deliberately deferred.

1.1 Source-of-truth order

When sources disagree, the agent must stop and apply this order:

1. protocol specification;
2. security, privacy, and wire invariants expressed by executable tests;
3. Rust reference architecture;
4. deployment profile;
5. this build plan;
6. issue text, comments, and local implementation convenience.

The agent may resolve a lower-level ambiguity without asking only when the choice is reversible, does not cross a trust boundary, and does not alter an agreed MVP decision. Otherwise it must record the alternatives and request human direction.

1.2 Meaning of completion

A work package is complete only when code, automated tests, user-facing behavior, operator documentation, and failure behavior agree. Compiling is necessary but not sufficient. A milestone is complete only when its demonstration can be run from a clean checkout using documented commands.

2 Decisions Already Made

The following choices are product inputs, not prompts for an implementation agent to revisit.

Decision	MVP commitment
Pilot audience	Invite-only deployment for approximately 20–50 cooperative users.
Primary product	Desktop application plus operator administration CLI.
Later clients	Native Swift and Kotlin applications reuse the Rust client core through a narrow FFI boundary.
Core journey	Bonded first contact, recipient decision, then decision-dependent future communication.
Value model	Nonredeemable test credits issued and inspected through authenticated administration.
Identity model	One protocol identity may enroll multiple explicit devices; every device has revocable key authority.
Recovery model	User-held recovery material plus trusted-device transfer; the provider cannot decrypt message content.
Initial desktop platform	macOS first; Windows follows after the pilot path is stable.
Deployment model	One provider, one region, and one PostgreSQL deployment.
Protocol scope	Native <i>cs-mail</i> ; no SMTP interoperability, federation, real payments, or mobile release in the MVP.
Local plaintext	Plaintext is not persisted by default. The desktop stores encrypted content and minimal metadata.
Pilot success	No lost protocol effects, duplicate settlement, provider-visible plaintext, or unrecoverable routine restart failure.

Exact retention durations remain a human policy decision. Milestone 1 must make them explicit before user data is created. Until then, the agent may build versioned retention mechanisms but must not invent final policy periods.

3 Architecture and Stability Rules

3.1 The hybrid organization

The existing repository is organized horizontally around protocol mechanics. That remains appropriate for the stable foundation. New product behavior is organized as vertical modules inside a small set of product crates.

```
crates/
  existing cs-mail-* crates      # stable horizontal foundation
  cs-mail-api-types              # versioned transport DTOs
  cs-mail-client-core            # platform-neutral client use cases
  cs-mail-mobile-ffi             # later milestone only

apps/
  cs-mail-server                 # testable API library + cs-maild binary
  cs-mail-admin-cli              # operator executable
  cs-mail-desktop                # Tauri shell + platform adapters
```

The new directories remain members of the existing Cargo workspace. Shared dependency versions and lints remain defined at the workspace root.

3.2 Dependency direction

Dependencies point inward. Existing stable crates must never depend on an application crate.

```
cs-mail-desktop ---> cs-mail-client-core ---> existing protocol/security/content
  crates
cs-mail-mobile-ffi --> cs-mail-client-core
cs-mail-admin-cli --> cs-mail-api-types
cs-mail-server ---> cs-mail-api-types + existing service/storage/worker crates
```

The API types crate must remain small. It may depend on serialization support and stable primitive identifiers, but it must not depend on Axum, Tauri, PostgreSQL, operating-system keychains, or a complete server implementation.

3.3 Vertical modules

Inside `cs-mail-client-core` and `cs-mail-server`, product behavior is grouped by user journey:

- **identity**: enrollment, address resolution, and profile state;
- **devices**: device keys, enrollment, listing, and revocation;
- **first_contact**: quote, reservation, content upload, and admission;
- **messages**: encrypted inbox, delivery state, and local decryption;
- **relationships**: accept, reject, block, unblock, and permission state;
- **recovery**: recovery material and trusted-device transfer; and
- **administration**: invites, test credits, policy, diagnostics, and migration status.

Each vertical owns orchestration, mapping to API types, product errors, and end-to-end tests. It reuses the protocol kernel; it does not restate transition rules.

3.4 Constraint on existing crates

The default is extension through new crates and additive APIs.

Layer	Examples	Change rule
Normative core	primitives, ledger, protocol, wire, security	Effectively frozen. Change only with human approval, a decision record, specification alignment, and conformance tests.
Implementation adapters	storage-postgres, service, worker, client, adapters	May evolve additively while preserving protocol meaning and existing public behavior.
Product layer	API server, admin CLI, client core, desktop, later mobile FFI	Expected location for MVP work.

No change may silently break a public Rust API, signed wire representation, stored protocol format, database migration history, settlement invariant, or privacy scope. Database migrations are forward-only after merge.

If an agent believes a normative-core change is necessary, it must first produce a short decision record containing:

1. the user-visible blocker;
2. evidence that an adapter or new crate cannot solve it cleanly;
3. the smallest proposed semantic change;
4. affected specification sections, vectors, migrations, and consumers;
5. compatibility and rollback consequences; and
6. new conformance tests that fail before the change and pass afterward.

3.5 When to create another crate

Create a crate only when at least one condition is true: two products reuse it; it isolates secrets, platform APIs, or heavy dependencies; it exposes an independently versioned contract; it produces a separate executable; or compile- time dependency enforcement materially improves safety. Otherwise create a module.

4 Cross-Cutting Engineering Contracts

These contracts apply to every milestone.

4.1 Security and trust

- Private content and device private keys remain at endpoints.
- Desktop secret keys use the operating-system keychain. They are never written to logs, configuration, SQLite, crash reports, or provider APIs.
- The provider accepts protocol commands only after canonical decoding, signature verification, scope verification, and durable key-authority checks.
- The admin CLI uses a distinct administrative authority. User command credentials cannot perform administrative operations.
- Sensitive values implement redacted debug output where accidental formatting would otherwise expose them.
- Every externally visible error is classified as safe to retry, terminal user action, authentication failure, or operator intervention.

4.2 API and versioning

The MVP uses HTTPS with versioned JSON resource DTOs for ordinary queries and opaque canonical signed command bytes for protocol mutations. The HTTP layer must not translate a signature-covered command into a semantically different representation.

The initial API surface is organized around these capabilities:

Capability	Client operation	Server responsibility
Enrollment	create identity and enroll device key	validate invite, create scoped identity, register device authority
Resolution	resolve recipient handle	return public identity and provider scope without private principal data
Commands	submit signed canonical command	verify, order, execute, persist, and return signed receipt
Content	upload and fetch encrypted record	enforce size, scope, lifetime, and authorization
Inbox	page encrypted delivery and solicitation summaries	expose cursor-stable minimized metadata
Relationships	query directed permission and decision state	project canonical state without inventing new semantics
Administration	issue invite, grant test credits, inspect health and policy	require separate admin authentication and audit

DTOs carry an explicit API version. Protocol version, wire version, policy version, privacy profile version, and retention policy version remain separate types and must never be inferred from the HTTP API version.

4.3 Client-core platform ports

The shared client core must not import Tauri, Swift, Kotlin, Keychain, Windows Credential Manager, SQLite bindings, or a concrete HTTP client in its domain modules. It defines narrow ports similar to the following illustrative shape:

```
pub trait KeyVault {
    fn create_device_keys(&self, identity: IdentityRef) -> Result<DeviceKeys, VaultError>;
    fn sign(&self, key: OperationalKeyRef, payload: &[u8]) -> Result<Signature, VaultError>;
    fn decrypt(&self, key: ContentKeyRef, record: &EncryptedContentRecord)
        -> Result<Vec<u8>, VaultError>;
}

pub trait LocalStore {
    fn save_ciphertext(&self, record: &EncryptedContentRecord) -> Result<(), StoreError>;
    fn save_cursor(&self, cursor: InboxCursor) -> Result<(), StoreError>;
}

pub trait ProviderTransport {
    fn submit_command(&self, command: SignedCommandBytes) -> Result<SignedReceipt, ApiError>;
    fn upload_content(&self, record: EncryptedContentRecord) -> Result<(), ApiError>;
}
```

Exact method names may change. The boundary is mandatory: platform adapters implement storage, transport, notification, and secret custody; use cases depend on traits and stable domain types.

4.4 Local data

The desktop may persist encrypted message records, minimized indexes, delivery cursors, relationship summaries, and nonsecret preferences in SQLite. Plaintext message bodies are decrypted for display and discarded when the view closes. Search over plaintext is out of scope for the MVP. A later explicit decision is required before introducing a local plaintext index.

4.5 Observability

Every service and CLI command uses structured tracing with a request or operation identifier. Logs may contain scoped public references, coarse status, durations, and typed error categories. They may not contain private principal mappings, message plaintext, private keys, full signed payloads, recovery material, or raw authentication evidence.

4.6 Repository-wide quality gate

Before every milestone handoff, the agent runs at least:

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace
```

PostgreSQL tests must also run in CI against an isolated database. Desktop smoke tests, API contract tests, migration-from-empty tests, and clean-install demo steps are added as their milestones introduce those surfaces.

5 Milestone Map

No.	Narrative outcome	Primary artifacts	Depends on
0	Protect the foundation	decision records, workspace skeleton, dependency rules	existing repository
1	The service comes alive	API server, administration CLI, explicit pilot policy	0
2	Alice gets an identity	shared client core, desktop shell, endpoint key custody	1
3	Alice makes first contact	quote, test-credit reservation, encryption, upload, admission	2
4	Bob's decision matters	solicitation inbox, accept/reject/block, settlement display	3
5	Time and failure become real	workers, retry, expiry, retention, restart recovery	4
6	A small pilot can use it	packaging, TLS, onboarding, recovery, operations	5
7	The core is ready for mobile	FFI boundary and mobile proof harness	6

6 Milestone 0: Protect the Foundation

Promise. An agent can add product code without destabilizing the protocol foundation or inventing a second architecture.

Allowed change. Workspace metadata, new directories, new empty crates, documentation, CI checks, and test-only dependency-graph validation. No protocol behavior changes.

Work packages

1. Create `crates/cs-mail-api-types`, `crates/cs-mail-client-core`, `apps/cs-mail-server`, `apps/cs-mail-admin-cli`, and `apps/cs-mail-desktop` as minimal workspace packages. A product package may contain a library target plus a thin binary target.
2. Add a short architecture decision record describing dependency direction, the existing-crate stability rule, and the core-change approval process.
3. Add automated dependency checks or a documented review command proving that stable crates do not depend on `apps/*`.
4. Define workspace-wide package metadata, lints, and dependency versions for newly introduced dependencies.
5. Add a root command or task runner entry that performs the complete quality gate without hiding individual Cargo commands.

Tests and failure cases

The workspace must compile with every new package present. Empty binaries must print version and help information. CI must fail if an application crate is added as a dependency of a stable core crate. Existing tests must remain unchanged and green.

Demonstration

From a clean checkout, list workspace packages, build the workspace, run the server and CLI help commands, and show the dependency-direction decision record.

Exit evidence. One clean CI run; crate skeleton committed; dependency graph reviewed; no normative-core diff.

Deferred. No API listener, database connection, Tauri window, enrollment, or message flow.

7 Milestone 1: The Service Comes Alive

Promise. An operator can start one provider deployment, initialize its database, inspect health, create invites, grant test credits, and view the active policy without editing SQL.

Allowed change. New API types, server, and CLI code; additive service/storage APIs when necessary; forward-only database migration. Normative semantics remain fixed.

Concrete defaults

Use Tokio for the process runtime, Axum for HTTP routing, Rustls for TLS, `tracing` for structured diagnostics, and Clap for the administration CLI. Keep the existing synchronous storage adapter behind a bounded blocking execution boundary for the first vertical slice; do not block an async executor worker directly. Replace that bridge with a TLS-configurable pool only when the behavior is covered by the same integration tests.

The CLI talks to an authenticated admin API for ordinary administration. A local server subcommand may apply migrations and perform first-deployment bootstrap. The CLI does not become a second application that writes arbitrary database rows.

Work packages

1. Define versioned health, policy, invite, test-credit, and error DTOs in `cs-mail-api-types`. Keep transport errors distinct from protocol errors.
2. Build `cs-mail-server` configuration with explicit bind address, database URL, deployment domain, provider identity, public base URL, content limits, and policy-version selection.
3. Add `cs-maild migrate`, `cs-maild bootstrap`, and `cs-maild serve`. Bootstrap must be idempotent and must not replace an existing key registry or provider identity.
4. Add authenticated admin endpoints for invite creation, test-credit grants, policy inspection, migration status, and health diagnostics.
5. Build matching CLI commands with machine-readable JSON output and concise human output. Mutating commands require explicit target display.
6. Decide and record exact pilot retention periods, content-size limits, invite lifetime, test-credit ceiling, and per-identity rate limits.
7. Add redaction tests for configuration and error output.

Mandatory tests

- migration from an empty PostgreSQL database and idempotent reapplication;
- refusal to start with missing provider keys or ambiguous deployment domain;
- admin authentication failure and separation from user command authority;
- duplicate invite and duplicate credit-grant idempotency;
- concurrent credit grants preserve ledger conservation; and
- health distinguishes process availability, database readiness, and migration readiness.

Demonstration

Start a fresh database, bootstrap the provider, start the API, create two invites, grant test credits to a seeded account, inspect balances and policy, restart the server, and show that bootstrap and credits were not duplicated.

Exit evidence. Documented clean-start transcript, API contract tests, PostgreSQL integration tests, redacted logs, and an operator quick-start requiring no direct SQL.

Deferred. User enrollment UI, message content, recipient decisions, SMTP, real payment rails, federation, multi-region deployment, and automated account recovery.

8 Milestone 2: Alice Gets an Identity

Promise. Alice installs the macOS desktop application, redeems an invite, creates a protocol identity and device keys, closes the application, and safely unlocks the same identity later.

Allowed change. Client-core use cases, identity/device API types and server routes, desktop shell, keychain and SQLite adapters, and additive service key-enrollment APIs.

Work packages

1. Define client-core ports for provider transport, key vault, local store, clock, and notification scheduling. Supply deterministic in-memory test adapters.
2. Implement invite redemption, identity enrollment, device-key creation, signed enrollment proof, identity resolution, device listing, and device revocation.
3. Implement the macOS key-vault adapter using the operating-system keychain. Store only key references in SQLite.
4. Create the local SQLite schema for identity profile, device metadata, encrypted content records, inbox cursors, minimized relationship summaries, and preferences. No plaintext body column is permitted.
5. Build a thin Tauri desktop shell with screens for welcome, invite redemption, identity creation, unlock, settings, and device list.
6. Make desktop commands call client-core use cases rather than protocol crates directly. UI components do not construct signed commands.
7. Add an explicit identity export/recovery placeholder that explains what is not yet recoverable. Do not imply provider-assisted content recovery.

Mandatory tests

- key material never appears in SQLite, logs, API payloads, or snapshots;
- duplicate invite redemption is idempotent for the same request and conflicts for another identity;
- a device key cannot enroll under another identity or provider scope;
- revoked devices cannot authenticate new commands after canonical revocation time;
- local schema upgrades preserve ciphertext and cursors; and
- close/reopen and simulated keychain denial produce safe, understandable behavior.

Demonstration

On a clean macOS user account, install the app, redeem Alice’s invite, create an identity, inspect the provider’s public enrollment record, quit, reopen, unlock, and list the enrolled device. Show that the database contains no private key or message plaintext.

Exit evidence. Signed development build, keychain integration test, local migration test, identity enrollment end-to-end test, and a screen-recordable demo path.

Deferred. Sending, inbox, recovery completion, second-device transfer, Windows adapter, background notifications, and mobile bindings.

9 Milestone 3: Alice Makes Encrypted First Contact

Promise. Alice resolves Bob, sees the quoted test-credit commitment, encrypts a message at her endpoint, submits bonded first contact, and receives durable provider evidence. The provider never receives plaintext.

Allowed change. First-contact and message verticals in client core and server; encrypted content and inbox API surfaces; additive adapters around existing content, security, service, and storage crates.

Work packages

1. Enroll Bob using the Milestone 2 flow and publish the required endpoint content-key certificate through an authenticated device operation.
2. Implement recipient resolution returning only provider-scoped public identity and active public encryption evidence.
3. Implement quote request and display. The UI shows processing charge, collateral, persistence reserve, total held value, expiry, and the difference between held, transferred, and refunded value.
4. Implement the client-core first-contact state machine: quote, validate, reserve, encrypt, upload ciphertext, admit, and persist signed receipt.
5. Preserve ordering: content may be uploaded before admission, but no recipient delivery becomes visible before durable admission.
6. Add an encrypted inbox feed with stable cursors and minimized metadata. Bob’s desktop fetches ciphertext and decrypts only for display.
7. Expose retry state without allowing a user to create a second economic effect from the same idempotency key.

Mandatory tests

- provider-side storage and logs contain no plaintext fixture fragments;
- ciphertext tampering, wrong recipient key, wrong relationship, and wrong declaration binding fail;
- insufficient test credits produce no hold, quote consumption, or message delivery;
- expired quote, expired content, and closed message-validity window settle or cancel exactly as specified;
- network failure after each step can be retried without duplicate reservation or admission; and
- Alice’s displayed balance and receipt match the canonical ledger and journal.

Demonstration

Alice selects Bob, types a message, reviews a quote, confirms first contact, and sees a pending decision. Bob refreshes his inbox and decrypts the message. An operator inspects provider records and demonstrates that only ciphertext, bounded metadata, signed evidence, and ledger effects are present.

Exit evidence. One automated Alice-to-Bob end-to-end test, one interrupted-network matrix, one plaintext-absence test, and a receipt/ledger reconciliation assertion.

Deferred. Recipient decisions, conversations, attachments, search, rich text, SMTP, external notifications, and real currency.

10 Milestone 4: Bob’s Decision Matters

Promise. Bob can understand and decide the solicitation. Acceptance enables future bond- free communication; rejection settles the attempt but preserves later eligible contact; blocking prevents ordinary future contact until Bob unblocks it.

Allowed change. Relationship use cases, solicitation UI, relationship projections, and receipt display. No new decision semantics.

Work packages

1. Build a solicitation inbox distinct from the ordinary accepted-message inbox. Group messages according to the existing relationship episode semantics.
2. Explain accept, reject, and block in plain language, including the immediate settlement consequence and future-contact consequence.
3. Submit recipient decisions as signed canonical commands and display the provider-signed receipt after durable commitment.
4. Add relationship status and a compact balance activity view for both users.

5. Implement accepted-message sending through the existing bond-free path. Do not simulate acceptance by issuing a zero-value quote.
6. Implement block and unblock management in settings, including the distinction between rejection history and reversible blocking.
7. Ensure all UI projections are rebuilt from canonical server state after restart or local cache deletion.

Mandatory tests

- acceptance settles every eligible open bond for the accepted public identity;
- acceptance does not forgive another public identity of the same principal;
- rejection preserves repeated-attempt backoff across public-identity rotation;
- block cancels unadmitted reservations and prevents later ordinary admission;
- concurrent decisions serialize to one canonical outcome; and
- the second accepted message is bond-free and still end-to-end encrypted.

Demonstration

Run three short stories: Bob accepts Alice and receives a second bond-free message; Bob rejects a fresh test identity and the balances settle visibly; Bob blocks and later unblocks another identity. Reopen both desktop apps and show the same canonical state.

Exit evidence. Decision matrix tests, concurrency test, projection-rebuild test, and reviewed UI copy explaining value movement without ambiguous words such as “fee” for a hold.

Deferred. Group communication, contact import, sophisticated abuse review, external funding, and federation.

11 Milestone 5: Time and Failure Become Real

Promise. Routine service failure, network interruption, duplicate requests, offline recipients, and elapsed deadlines do not lose messages, duplicate settlement, or strand value.

Allowed change. Worker executable modes, leases, operational scheduler authority, notification polling, retention execution, pooling/TLS improvements, and operational tests.

Work packages

1. Run outbox delivery, scheduled deadlines, lane horizons, and retention deletion as supervised worker loops in `cs-maild` or a separate worker binary from the same server package.
2. Configure a distinct scheduler operational key and prove that only opaque, live database claims can trigger unsigned system-derived commands.

3. Add bounded exponential retry with typed transient and terminal errors. Protocol idempotency, not timing luck, prevents duplicate effects.
4. Add TLS-configurable PostgreSQL connections and a bounded connection pool without changing storage semantics.
5. Implement desktop inbox polling and local notifications that reveal no message plaintext on a locked screen by default.
6. Execute retention rules and emit deletion manifests. Protect ciphertext still required by an undelivered outbox item.
7. Add graceful shutdown: stop accepting new work, finish or release claims, flush logs, and close database connections within a bounded time.

Mandatory tests

- terminate the service after each durable boundary and recover correctly;
- reclaim expired worker leases without double acknowledgement;
- reject forged, foreign, expired, or wrong-scheduler claims;
- materialize admission timeout, bond expiry, and persistence release exactly once;
- preserve undelivered ciphertext through retention and delete it afterward;
- tolerate recipient offline periods and cursor replay; and
- run a sustained concurrency test with invariant checks on ledger conservation and journal order.

Demonstration

Pause Bob’s desktop, send a message, kill the worker during delivery, restart the service, reconnect Bob, and show one recipient-visible message and one canonical economic outcome. Run a short-deadline attempt and show automatic cancellation or expiry plus the correct balance movement.

Exit evidence. Failure-injection report, lease tests against PostgreSQL, graceful-shutdown test, retention test, TLS database configuration, and a documented recovery procedure.

Deferred. Multi-region failover, high-availability control plane, mobile push delivery, large-scale load targets, and external incident automation.

12 Milestone 6: A Small Pilot Can Use It

Promise. A small invited group can install, understand, use, recover, and receive support for the product without a developer manually repairing normal state.

Allowed change. Packaging, code signing, onboarding polish, recovery completion, multi-device enrollment, diagnostics, backups, pilot operations, and Windows adapter work after the macOS path is stable.

Work packages

1. Package and sign the macOS application. Produce deterministic server and CLI release artifacts with embedded version information.
2. Complete user-held recovery material generation, confirmation, secure display, and restoration. Recovery must not grant the provider content decryption authority.
3. Implement trusted-device enrollment: an existing device authorizes a new device, users can list devices, and revocation has canonical time semantics.
4. Add onboarding explanations for test credits, first-contact holds, recipient decisions, encryption, recovery responsibility, and device loss.
5. Add safe diagnostics export containing versions, coarse timings, typed failures, and request references but no secrets or plaintext.
6. Document backups and conduct restore exercises for PostgreSQL, configuration, provider public authority, and separately protected provider private keys.
7. Define pilot support, abuse-report intake, account deletion, retention, incident escalation, and release rollback procedures.
8. Add Windows key-vault, installer, notification, and local-store adapters only after the macOS acceptance suite is platform-neutral.

Mandatory tests

- clean install, upgrade, uninstall, and reinstall on supported macOS versions;
- recovery from user-held material on a new clean device;
- trusted-device addition and immediate revoked-device refusal;
- backup restore followed by receipt, balance, relationship, and inbox reconciliation;
- diagnostics export secret scan; and
- pilot-scale soak test with 50 identities, offline intervals, decisions, retries, and restarts.

Demonstration

Give two nondeveloper testers clean machines and only the pilot documentation. They install, enroll, fund through an operator, exchange first contact, decide, send again, add or recover a device, and export safe diagnostics. The operator restores a staged backup and reconciles the canonical state.

Exit evidence. Signed installers, release manifest, recovery ceremony transcript, restore report, support runbook, pilot policy, and human usability findings with critical issues resolved.

Deferred. Public signup, production financial value, SMTP, federation, mobile release, large attachments, rich collaboration, and broad compliance automation.

13 Milestone 7: The Core Is Ready for Mobile

Promise. The product’s protocol and client behavior can move into native mobile applications without copying business logic or weakening endpoint key custody.

Allowed change. Client-core portability improvements, a narrow FFI crate, generated bindings, and a proof harness. No production mobile application is required.

Work packages

1. Audit `cs-mail-client-core` for desktop, filesystem, threading, and UI assumptions. Replace them with explicit ports where necessary.
2. Define an FFI-safe facade using bounded value types, stable error codes, cancellation, and explicit memory ownership. Do not expose internal Rust structs or cryptographic secret bytes as a convenience API.
3. Generate Swift and Kotlin bindings for identity summary, inbox page, message decryption request, first-contact quote, send progress, decision, and device management.
4. Build mock iOS and Android adapter harnesses for secure-key operations, transport, local ciphertext storage, and notifications.
5. Run the same client-core conformance scenarios through direct Rust, Swift binding, and Kotlin binding entry points.
6. Document platform responsibilities: Keychain or Keystore custody, biometric policy, background notification handling, screen privacy, and recovery-material presentation.

Mandatory tests

- deterministic behavior parity across Rust, Swift, and Kotlin harnesses;
- no secret value crosses the FFI boundary except into an approved platform key operation;
- cancellation and process termination do not leave a half-recorded local operation;
- malformed or oversized FFI input returns typed errors without panic; and
- bindings build in clean platform CI environments.

Demonstration

From a minimal Swift harness and a minimal Kotlin harness, unlock a test identity through a mock secure-key adapter, fetch one encrypted message, decrypt through the Rust core, and return a display-safe result. Show identical receipt and error interpretation on both platforms.

Exit evidence. Versioned FFI surface, generated bindings, parity suite, platform responsibility document, and no Tauri dependency in the client core or mobile FFI graph.

Deferred. App Store and Play Store releases, production push notifications, polished mobile UI, mobile account recovery ceremony, and tablet-specific design.

14 Agent Execution Protocol

An agent assigned a milestone must work in thin, demonstrable slices. It should not implement every server route before connecting one end-to-end user action.

14.1 Required loop for each work package

1. **Inspect.** Read applicable specifications, existing APIs, tests, migrations, and local repository instructions.
2. **State the slice.** Name one user-visible outcome and its exact acceptance evidence.
3. **Map impact.** List new crates/modules, additive existing-crate APIs, migrations, trust boundaries, and consumers.
4. **Implement inward-out.** Start with contract tests and domain use case, then adapters, then transport, then UI.
5. **Test failure first.** Cover replay, expiry, wrong scope, interruption, concurrency, and redaction relevant to the slice.
6. **Demonstrate.** Run the smallest real path using actual binaries and PostgreSQL where the slice crosses persistence.
7. **Handoff.** Report behavior delivered, evidence, remaining risks, deferred work, and any decision requiring a human.

14.2 Prohibited shortcuts

An agent must not:

- bypass signed durable execution to simplify tests or UI integration;
- persist plaintext or private keys because secure storage is incomplete;
- make the UI reconstruct protocol settlement rules;
- write directly to canonical tables from the admin CLI;
- reuse protocol, wire, policy, privacy, and API version numbers interchangeably;
- silently rewrite an applied migration;
- treat ignored PostgreSQL tests as evidence that database behavior passed;
- introduce real monetary value or SMTP scope without a new human decision; or
- declare a milestone complete without its clean-checkout demonstration.

14.3 Decision record threshold

The agent may choose names, internal module organization, reversible UI details, query structure, test fixtures, and comparable libraries within the fixed architecture. It must request approval before changing identity/device meaning, recovery authority, retention policy, value semantics, supported platform order, deployment topology, cryptographic suite, wire format, protocol state, or the existing-crate stability rule.

15 End-to-End Acceptance Matrix

The following matrix is cumulative. A later milestone must continue to pass all earlier rows.

Journey	Introduced	Success evidence	Critical failure assertion
Fresh deployment	M1	migrate, bootstrap, serve, health, restart	no duplicate authority or migration
Invite and test credit	M1	CLI creates invite and auditable credit	no direct arbitrary ledger write
Identity enrollment	M2	reopenable endpoint identity and device list	no key in provider or SQLite
Bonded first contact	M3	Bob decrypts; Alice has receipt and correct hold	provider has no plaintext; replay has no second hold
Acceptance	M4	second message is encrypted and bond-free	other public identity remains unaccepted
Rejection and retry	M4	settlement and later eligible quote reflect history	identity rotation does not reset attempt history
Block and unblock	M4	ordinary admission refused until unblock	unblocking does not erase history
Worker recovery	M5	one outcome after crash and lease reclaim	no duplicate delivery acknowledgement or settlement
Retention	M5	eligible ciphertext deleted with manifest	undelivered ciphertext not deleted early
Recovery and device transfer	M6	clean device regains authorized access	provider still cannot decrypt
Mobile binding parity	M7	Swift/Kotlin harness matches Rust behavior	no secret leakage or panic across FFI

16 Risk Register and Stop Conditions

Risk	Early control	Stop condition
Core semantic drift	frozen-core rule, decision records, conformance tests	product work requires an unexplained protocol change
Secret leakage	key-vault port, redaction, plaintext-absence tests	key or plaintext appears in server, logs, SQLite, or diagnostics
Duplicate economics	signed idempotent commands and failure matrix	retry produces a second hold, transfer, or decision
Misleading UX	reviewed quote and decision copy tied to ledger evidence	UI describes held value as transferred or hides settlement consequence
Operational fragility	restart, lease, backup, and restore tests	clean restart or restore cannot reconcile journal, ledger, and inbox
Crate proliferation	crate threshold and inward dependencies	feature crates duplicate domain types or create cycles
Premature platform spread	macOS-first exit gate	Windows or mobile work delays the first complete macOS journey
Policy ambiguity	versioned explicit pilot policy	user data would be created without retention and deletion periods

If a stop condition occurs, the agent must preserve evidence, avoid broad workarounds, and request human direction. It must not weaken the test that exposed the condition.

17 MVP Completion Definition

The desktop MVP and administration CLI are complete when all of the following are true:

1. a clean deployment can be migrated, bootstrapped, operated, backed up, restored, and diagnosed without direct database edits;
2. two clean macOS installations can enroll independent identities and retain private keys only through endpoint-controlled custody;
3. Alice can make encrypted bonded first contact and Bob can decrypt it;
4. Bob can accept, reject, block, and unblock with correct visible and canonical settlement effects;
5. accepted follow-up communication is bond-free and encrypted;
6. retries, duplicates, restarts, offline delivery, expiry, and retention pass the cumulative acceptance matrix;
7. recovery and trusted-device transfer work without granting provider content access;
8. the signed installer, server, worker, and CLI have documented release and rollback procedures;
9. the pilot policy states retention, test-credit, limits, abuse, support, and account-deletion behavior; and
10. a nondeveloper can complete the primary journey from the documentation.

This completion definition intentionally excludes real payments, SMTP, federation, public signup, multi-region operation, and a production mobile app. Those are later product programs, not hidden requirements for declaring the native desktop pilot successful.

18 Recommended First Agent Assignment

The first assignment should be Milestone 0 followed by only the walking skeleton of Milestone 1. A suitable bounded objective is:

Create the agreed product-crate skeleton, enforce inward dependency direction, and deliver a server binary plus administration CLI that can report version and health against a migrated development database. Do not add enrollment, content, or message behavior. Preserve every existing test and make no normative-core change. Finish with clean-checkout commands, automated tests, and a concise decision record describing the new crate boundaries.

That assignment proves the build plan itself: the agent must create a small, working vertical path while respecting the architecture and stability constraints before it receives authority to build user identity or messaging.